

Interim Proposal: GraphModi

Arihant Rastogi

2023111012

Prasoon Dev

2023111014

Yajat Lakhanpal

2023111015

Hrishiraj Mitra

2023111037

Team Hansraj

International Institute of Information Technology, Hyderabad

Abstract

Graph Language Models (GLMs) fuse a graph encoder with a language model so that answers are conditioned on an input graph G . Existing work treats G as fixed. That breaks as soon as the setting is conversational: if a user revises a fact in language, the encoder still reads the old graph, and later answers can be grounded in a stale world. We do not propose a new architecture. We wrap an existing GLM in a loop: the model emits a small edit, we apply it to G , and we re-encode before the next question. We evaluate on a small synthetic dataset where the new answer is only available from the updated graph. We compare a frozen graph, an oracle-updated graph, prompting the revision in text, a GraphRAG-style transcript baseline, and the self-edit loop, scoring answer exact match and whether the parsed edit is correct.

1 Introduction

A Graph Language Model answers questions from an input graph G . The usual setup is a single pair (G, q) . In a conversation, a user may correct a fact. The language model sees the correction, but the graph encoder still reads the original G . Later answers can then be grounded in a stale graph.

Plenz and Frank (2024) introduced GLMs so that structure and text are encoded together. He et al. (2024) describe a conversational interface (“chat with your graph”), but GraphQA is still single-shot. Their ablations show that both the GNN path and the textualized graph matter: if conversation revises a fact and G does not, retrieval from the old graph grounds the model in the wrong world. TEA-GLM (Wang et al., 2024), GraphToken (Perozzi et al., 2024), and related models likewise treat G as a fixed dataset. CLEGR (Petkar et al., 2026) shows that many GLM benchmarks can be solved from one modality. Agentic Graph Token reasoning (Peng and Yang, 2026) lets a model re-choose a view of a still-fixed graph. None of these settings

test a user correction that makes G stale.

Conversational KG and GraphRAG systems keep state in an external store or retrieve serialized triples for an LLM. That is the right design if G is a database. It is not the GLM setting: graph tokens remain part of the forward pass and are not overwritten by extra retrieved text. Auditors such as SKG-Eval (Shil and Samui, 2026) score dialogue against a parallel KG. They do not update the GLM’s input graph. Tool-using LLMs call graph algorithms; G-Retriever’s retrieval is pipeline code. None of these expose a mutation of the tensors the GNN encodes.

The claim is simple. GLMs exist so answers are grounded in G . Chat includes corrections. There is currently no write path that refreshes the graph the encoder reads. Two turns of inconsistent input are enough. We are not adding a memory model or native tool calling.

2 Methodology

A GLM forward pass cannot rewrite G inside the GNN. We put an outer loop around an existing model (for example G-Retriever, GraphToken, or TEA-GLM).

On each user turn u_t :

1. Prompt the GLM with the current graph and u_t to emit an edit Δ .
2. Parse Δ . If it is illegal, treat it as a no-op.
3. Otherwise apply Δ to G with a small schema-checked Python function.
4. Re-encode G and answer the next question q_t .

The model has no tools. It already emits text. We restrict that text to four operations: SET a node attribute, DEL an edge, ADD an edge, or NOOP.

Two calls, same weights: emit Δ , then answer on the updated G . The QA prompt is only (G, q_t) .

The revision utterance is not passed to QA. If QA succeeds, the fact is in the graph. We start with prompting and parsing. Constrained decoding or light LoRA on the same LLM head is only if writing the edit is the bottleneck. No second model.

If we skip re-encoding, this is just GraphRAG. It is a GLM method only if the apply step always precedes the next GNN forward pass.

3 Dataset

We will not scrape a dialogue corpus. Items are generated from a synthetic attributed graph, in the style of CLEGR: a fictional metro with invented station names, lines, and attributes, so parametric memory cannot skip G .

The generator samples a small graph G_0 (about 15 to 30 nodes) with attributes such as open or closed, and at least two paths between a query pair so that closing a node changes the answer. It fixes a query that can be executed on the graph, for example the number of stations on the shortest path using only open stations. It then samples a short sequence of SET, ADD, and DEL operations that change that answer, and verbalizes each operation as a short user utterance. Gold answers are computed by a solver on the graph, not written by hand.

We discard an item if the question can be answered without the graph, if the utterance plus question recover the answer with no graph, or if the original G_0 already yields the new answer. A few hundred sessions of two to five turns is enough to run the comparisons below.

4 Benchmarking

The point of the dataset is to ask whether the GLM has to update G , or whether text is already enough. We therefore compare a few conditions on the same sessions.

A frozen graph should keep the old answer after a revision. If it does not, the item is too easy or the model is not using G . Applying the gold edit and re-encoding, then asking q_t with no revision text, should give the new answer. That checks that the GLM can read an updated graph. If instead we leave G_0 frozen and just prompt the revision in text, and that already matches the oracle, then there is nothing to do: prompting suffices and rewriting G is unnecessary.

The method we care about is the self-edit loop. The model writes Δ , we apply it, and we ask q_t on the new graph, again without passing u_t into

QA. We also compare a GraphRAG-style baseline that keeps the transcript and does not re-encode. We measure exact match on answers against the solver, and whether the parsed edit matches the gold edit. If oracle updates restore accuracy and the self-edit loop does not, the bottleneck is writing Δ , not reading G .

References

- Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V. Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. [G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering](#). In *Advances in Neural Information Processing Systems*.
- Zhuoyi Peng and Yi Yang. 2026. [Agentic graph token reasoning](#). *Preprint*, arXiv:2608.00542.
- Bryan Perozzi, Bahare Fatemi, Dustin Zelle, Anton Tsitsulin, Mehran Kazemi, Rami Al-Rfou, and Jonathan Halcrow. 2024. [Let your graph do the talking: Encoding structured data for LLMs](#). *Preprint*, arXiv:2402.05862.
- Soham Petkar, Hari Aakash K, Anirudh Vempati, Akshit Sinha, Ponnurangam Kumaraguru, and Chirag Agarwal. 2026. [A graph talks, but who’s listening? rethinking evaluations for graph-language models](#). In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 32441–32462, San Diego, California, United States. Association for Computational Linguistics.
- Moritz Plenz and Anette Frank. 2024. [Graph language models](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4477–4494, Bangkok, Thailand. Association for Computational Linguistics.
- Avijit Shil and Suman Samui. 2026. [SKG-Eval: Stateful evaluation of multi-turn dialogue via incremental semantic knowledge graphs](#). *Preprint*, arXiv:2605.16650.
- Duo Wang, Yuan Zuo, Fengzhi Li, and Junjie Wu. 2024. [LLMs as zero-shot graph learners: Alignment of GNN representations with LLM token embeddings](#). In *Advances in Neural Information Processing Systems*.